



Politechnika
Wrocławska

Wroclaw University of Science and Technology
Faculty of Information and Communication Technology
Department of Control Systems and Mechatronics

Design And Application of Neural Network (Project)

Title: Neural Text Scorer

Aleksander Żołnowski - 272536

Kamil Fiącek - 272612

Submission Date 19.06.2025

June 2025
Wrocław, Poland

1 Introduction

Automatic evaluation of written texts is a key challenge in the field of Natural Language Processing (NLP), with applications in education, recruitment, and writing assistance tools. The goal is to analyze a piece of writing and assign a quality score similar to that given by a human evaluator, considering multiple aspects such as language correctness, readability and general score. Manual grading is time-consuming, subjective, and costly, which has led to increasing interest in AI-based solutions for this task.

In this project, we investigate how different neural network architectures perform on the task of assessing the quality of texts such as technical reports, essays, and written assignments. Specifically, we compare the performance of Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), and Recurrent Neural Network (RNN) models. These architectures represent fundamentally different approaches to processing textual data, and the aim of this study is to evaluate which of them is best suited for the task of automated text assessment based on multiple quality criteria.

2 Problem

The task is to compare different learning models that can predict the overall quality score of a text based on its content. The evaluation is based on 3 main criteria:

1. **Language correctness** – grammar, spelling, punctuation,
2. **Readability** - clear organization and logical flow of paragraphs and sentences,
3. **General score** - General quality score of the text

The models we compare include:

- **MLP** - Multilayer Perceptron
- **CNN** - Convolutional Neural Network
- **RNN** - Recurrent Neural Network

The objective of this project is to perform an empirical comparison of these three models, evaluate their effectiveness on the task of text scoring, and derive conclusions about their strengths and limitations in assessing different dimensions of text quality.

3 Objective

3.1 General Objective

The main objective of our project was to develop and evaluate a system for assessing texts using three different neural networks, and then compare each networks' results. In the last step we prepared an user interface, which takes a text as the input and assesses it on the fly with practical visualization.

3.2 Specific Objectives

We utilized three data sets, each one to assess different properties:

- ASAP 2.0 - content quality evaluation
- CommonLit Text Features - readability and text quality assessment
- JFLEG - grammar errors detection

The networks were trained with different parameters to see their impact on the final rating.

4 Literature Review/Related Work

- Goodfellow, I., Bengio, Y., Courville, A. (2018). *Deep Learning*.
- Chollet, F. (2018). *Deep Learning with Python*.
- <https://arxiv.org/abs/1804.07954>

- <https://arxiv.org/abs/1907.11779>
- <https://arxiv.org/abs/1805.12471>
- <https://arxiv.org/abs/1606.04289>
- <https://www.tensorflow.org> (documentation)
- <https://www.kaggle.com> (data)

5 Methodology

5.1 Data Acquisition

The datasets were acquired from [kaggle.com](https://www.kaggle.com):

- ASAP 2.0: <https://www.kaggle.com/datasets/lburleigh/asap-2-0>
- CommonLit: <https://www.kaggle.com/datasets/douglaskgaraujo/commonlit-text-features>
- JFLEG: <https://www.kaggle.com/datasets/thedevastator/jfleg-english-grammatical-error-benchmark>

5.2 Preprocessing Techniques

To prepare the data for training and evaluation, we performed several preprocessing steps involving data cleaning, tokenization, padding, feature scaling and formatting for multi-task learning. Below are the main steps of the preprocessing:

1. Data Loading and Cleaning - We use three datasets for different but related tasks:

- **ASAP** for essay scoring
- **CommonLit** for readability prediction
- **JFLEG** for grammatical error detection

Each dataset was loaded from CSV files. In the case of JFLEG, which provides corrected versions of grammatically incorrect sentences, we extracted the first available correction (if any) and created a balanced dataset consisting of both erroneous and corrected sentences. This allowed us to formulate the grammatical error detection task as a binary classification problem (`has_errors` = 1 for original, 0 for corrected). Missing or null text entries were removed, and all text fields were converted to string type to ensure consistency.

2. Tokenization

We used Keras's Tokenizer to convert raw text into sequences of integer indices. A shared tokenizer was fit on the combined text data from all three datasets (ASAP essays, CommonLit excerpts, and JFLEG sentences). We limited the vocabulary size to a maximum of 20,000 tokens and used `UNK` for out-of-vocabulary words.

3. Padding

To standardize input lengths across all datasets, token sequences were padded to a fixed length of 300 tokens using post-padding. This ensures that the models receive uniformly sized inputs regardless of the original text length.

4. Feature Scaling

For the CommonLit dataset, the `flesch_reading_ease` scores were normalized to the [0, 1] range using `MinMaxScaler`. This scaling facilitates regression learning by bounding the target variable and improving numerical stability.

5. Multi-Task Formatting

The processed data from the three datasets were merged into a single dataframe with the following structure

- **X** - tokenized and padded sequences
- **y_asap** - target score for ASAP
- **y_commonlit** - scaled readability score for CommonLit

- **y_jfleg** - grammatical correctness score

Missing labels (due to multi-task setup) were filled with zeros, and separate sample weight masks were generated to ensure that each model head is trained only on relevant data. This enables the use of a unified multi-task model with shared inputs but multiple outputs.

6. Dataset Splitting

The final combined dataset was randomly split into training(80%), validation(10%) and test(10%) sets using `train_test_split` with a fixed random seed (42). Each split preserves the multi-task structure, with inputs **X**, targets **y**, and sample weight masks for each task.

5.3 Implementation Details

5.3.1 MLP

The MLP-based model follows a multi-task learning architecture with a shared feature extractor and three task-specific output heads. It processes fixed-length tokenized text sequences of length 300 using a vocabulary of the top 20,000 most frequent tokens.

5.3.1.1 Shared Feature Extractor The model starts with an embedding layer of dimension 256, using `mask_zero=True` to ignore padding tokens. The embedded input is regularized with dropout (rate 0.2) and then passed through two pooling operations: global average pooling and global max pooling. These two representations are concatenated and form the input to the shared fully connected layers. The shared trunk consists of three dense layers:

- Dense(512) with ReLU activation, L2 regularization(0.001), followed by batch normalization and dropout (0.4)
- Dense(256) with ReLU, L2, batch normalization and dropout (0.3)
- Dense(128) with ReLU, L2, batch normalization and dropout (0.3)

These layers serve as a task-independent encoder capturing general features from the text.

5.3.1.2 Task-Specific Heads The output from the shared trunk is passed to three separate heads:

- **ASAP (Essay Scoring):**
 - Dense(64) → BatchNorm → Dropout(0.2)
 - Dense(32)
 - Output: Dense(1) (regression)
- **CommonLit (Readability Regression):**
 - Dense(64) → BatchNorm → Dropout(0.2)
 - Dense(32)
 - Output: Dense(1) (regression)
- **JFLEG (Grammaticality Classification):**
 - Dense(128) → BatchNorm → Dropout(0.4)
 - Dense(64) → BatchNorm → Dropout(0.3)
 - Dense(32) → BatchNorm → Dropout(0.2)
 - Output: Dense(1) with sigmoid activation (binary classification)

5.3.1.3 Training and Optimization The model is trained using the Adam optimizer with a learning rate of 0.001 and gradient clipping (`clipnorm = 1.0`) for stability. A custom multi-task loss function is used with the following components:

- Mean Squared Error (MSE) for ASAP and CommonLit (regression),
- Binary Crossentropy for JFLEG (classification).

Loss weights were selected to balance task importance:

- `score_output`: 15.0,
- `readability_output`: 1.0,
- `jfleg_output`: 8.0.

Sample weights were also passed to each task during training to account for label quality and class imbalance. Training used the following callbacks:

- Early stopping (patience = 10, min delta = 0.005),
- ReduceLROnPlateau (factor = 0.3, patience = 4),
- Learning rate scheduler: epochs 10–19: $\text{lr} \cdot 0.9$, epochs ≥ 20 : $\text{lr} \cdot 0.8$,
- ModelCheckpoint monitored validation accuracy on JFLEG.

5.3.2 CNN

The CNN-based model follows a multi-task learning architecture with a shared feature extractor and three task-specific output heads. It processes fixed-length tokenized text sequences of length 300 using a vocabulary of the top 20,000 most frequent tokens.

5.3.2.1 Shared Feature Extractor The model starts with an embedding layer of dimension 256, using `mask_zero=True` to ignore padding tokens. The embedded input is regularized with `SpacialDropout1D` (rate 0.2) and then passed through three parallel convolutional layers:

- `Conv1D(128, kernel_size=3, activation='relu', padding='same')`
- `Conv1D(128, kernel_size=5, activation='relu', padding='same')`
- `Conv1D(128, kernel_size=7, activation='relu', padding='same')`

Pooling operations are applied to each convolutional output:

- Global average pooling on the first,
- Global max pooling on the second,
- Global max pooling (simulating sum pooling) on the third.

These pooled outputs are concatenated and serve as input to the shared fully connected layers:

- `Dense(512)` with ReLU activation, L2 regularization(0.001), followed by batch normalization and dropout (0.4)
- `Dense(256)` with ReLU, L2, batch normalization and dropout (0.3)
- `Dense(128)` with ReLU, L2, batch normalization and dropout (0.3)

These layers serve as a task-independent encoder that captures general-purpose features from the input.

5.3.2.2 Task-Specific Heads The output of the shared trunk is passed to three separate task-specific heads:

- **ASAP (Essay Scoring):**
 - `Dense(64) → BatchNorm → Dropout(0.2)`
 - `Dense(32)`
 - Output: `Dense(1)` (regression)
- **CommonLit (Readability Regression):**
 - `Dense(64) → BatchNorm → Dropout(0.2)`
 - `Dense(32)`
 - Output: `Dense(1)` (regression)
- **JFLEG (Grammaticality Classification):**
 - `Dense(128) → BatchNorm → Dropout(0.4)`
 - `Dense(64) → BatchNorm → Dropout(0.3)`
 - `Dense(32) → BatchNorm → Dropout(0.2)`
 - Output: `Dense(1)` with sigmoid activation (binary classification)

5.3.2.3 Training and Optimization The model is trained using the Adam optimizer with a learning rate of 0.001 and gradient clipping (`clipnorm = 1.0`) for stability. A custom multi-task loss function is used with the following components:

- Mean Squared Error (MSE) for ASAP and CommonLit (regression),
- Binary Crossentropy for JFLEG (classification).

Loss weights are applied to balance task importance:

- `score_output`: 15.0,
- `readability_output`: 1.0,
- `jfleg_output`: 8.0.

Sample weights are passed per task to account for label quality and class imbalance. Standard callbacks are used during training:

- EarlyStopping (patience = 10, min_delta = 0.005)
- ReduceLROnPlateau (factor = 0.3, patience = 4)
- Learning rate scheduler: epochs 10–19: $\text{lr} \cdot 0.9$, epochs ≥ 20 : $\text{lr} \cdot 0.8$
- ModelCheckpoint monitoring JFLEG validation accuracy

5.3.3 RNN

The RNN-based model also follows a multi-task learning architecture with a shared feature extractor and three task-specific heads. It processes fixed-length tokenized text sequences of length 300 using a vocabulary of the top 20,000 most frequent tokens.

5.3.3.1 Shared Feature Extractor The model starts with an embedding layer of dimension 256, using `mask_zero=True`. The embedded input is regularized with `SpatialDropout1D` (rate 0.3) and then passed through bidirectional LSTM: `Bidirectional(LSTM(64, return_sequences=True))`.

The LSTM output is aggregated using `GlobalMaxPooling1D`. The pooled output is then passed through two fully connected layers:

- `Dense(128)` with ReLU, `L2(0.001)`, followed by batch normalization and dropout (0.4)
- `Dense(64)` with ReLU, `L2(0.001)`, followed by batch normalization and dropout (0.3)

These layers form the shared encoder used by all tasks.

5.3.3.2 Task-Specific Heads The shared output is fed into three individual task heads:

- **ASAP (Essay Scoring):**
 - `Dense(32) → Dropout(0.2)`
 - Output: `Dense(1)` (regression)
- **CommonLit (Readability Regression):**
 - `Dense(32) → Dropout(0.2)`
 - Output: `Dense(1)` (regression)
- **JFLEG (Grammaticality Classification):**
 - `Dense(64) → Dropout(0.3)`
 - `Dense(32)`
 - Output: `Dense(1)` with sigmoid activation (binary classification)

5.3.3.3 Training and Optimization The model is trained using the Adam optimizer (lr=0.001, clipnorm=1.0) and a multi-task loss function with:

- MSE for ASAP and CommonLit
- Binary Crossentropy for JFLEG

Loss weights are as follows:

- score_output: 5.0,
- readability_output: 1.0,
- jfleg_output: 8.0.

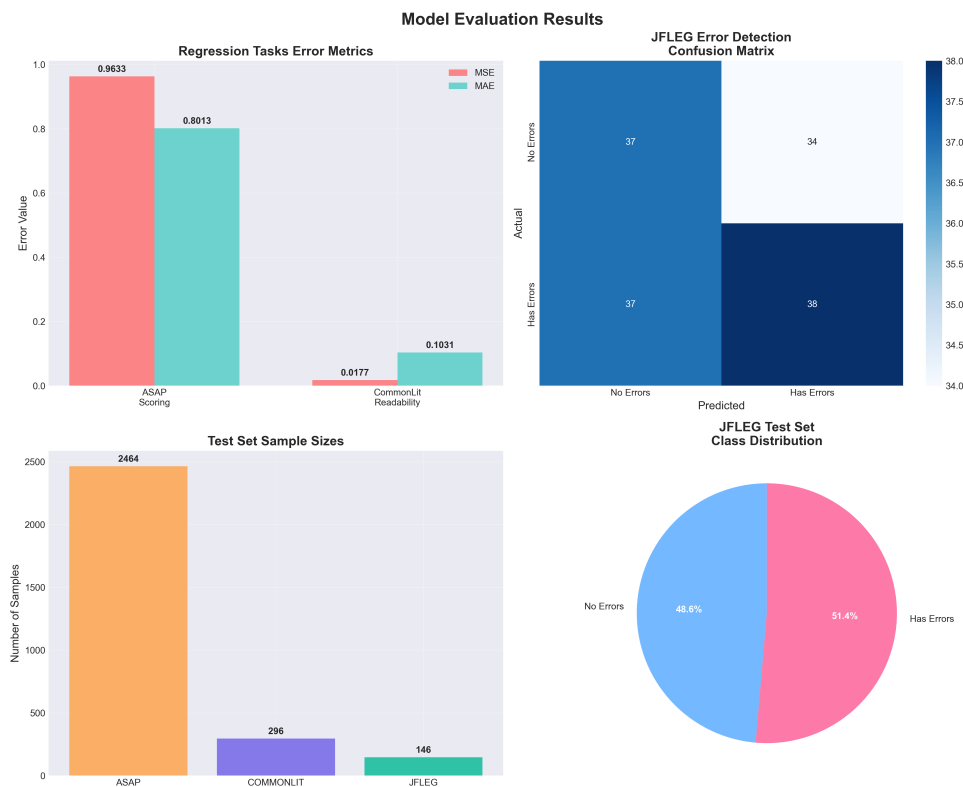
Sample weights are passed per task to handle label noise and class imbalance.

Standard callbacks like EarlyStopping, ReduceLROnPlateau, dynamic learning rate scheduling, and ModelCheckpoint can be used similarly to the CNN and MLP models.

5.4 Evaluation Metrics

5.4.1 MLP

The evaluation was conducted for three separate tasks using the multi-head MLP model: essay scoring (ASAP), readability prediction (CommonLit), and grammatical error detection (JFLEG). Each task was assessed using task-appropriate metrics: regression metrics for ASAP and CommonLit, and classification metrics for JFLEG.



Rysunek 1: MLP Evaluation Results

ASAP (Essay Scoring – Regression Task) The model performed relatively well in predicting essay grades:

- Number of samples: 2464
- Mean Squared Error (MSE): 0.9633
- Mean Absolute Error (MAE): 0.8013
- Root Mean Squared Error (RMSE): 0.9815

- Score range: 1.00 -- 6.00

The error values indicate that the model makes an average error of about one rating point, which is acceptable for a basic MLP model but could be improved with more complex architectures.

CommonLit (Readability Prediction – Regression Task) For the readability level regression, the model achieved very good prediction quality:

- Number of samples: 296
- Mean Squared Error (MSE): 0.0177
- Mean Absolute Error (MAE): 0.1031
- Root Mean Squared Error (RMSE): 0.1330
- Readability range: 0.04 -- 0.96

Low error values indicate high model accuracy in this task, which may be due to more predictable language patterns related to readability.

JFLEG (Grammatical Error Detection – Classification Task) In the classification task, the model had difficulty correctly distinguishing between grammatically correct and grammatically incorrect sentences:

- Number of samples: 146
- Accuracy: 0.5137
- Class distribution:
 - No errors (label 0): 71
 - Has errors (label 1): 75

Confusion Matrix:

	Predicted: No Errors	Predicted: Has Errors
Actual: No Errors	37	34
Actual: Has Errors	37	38

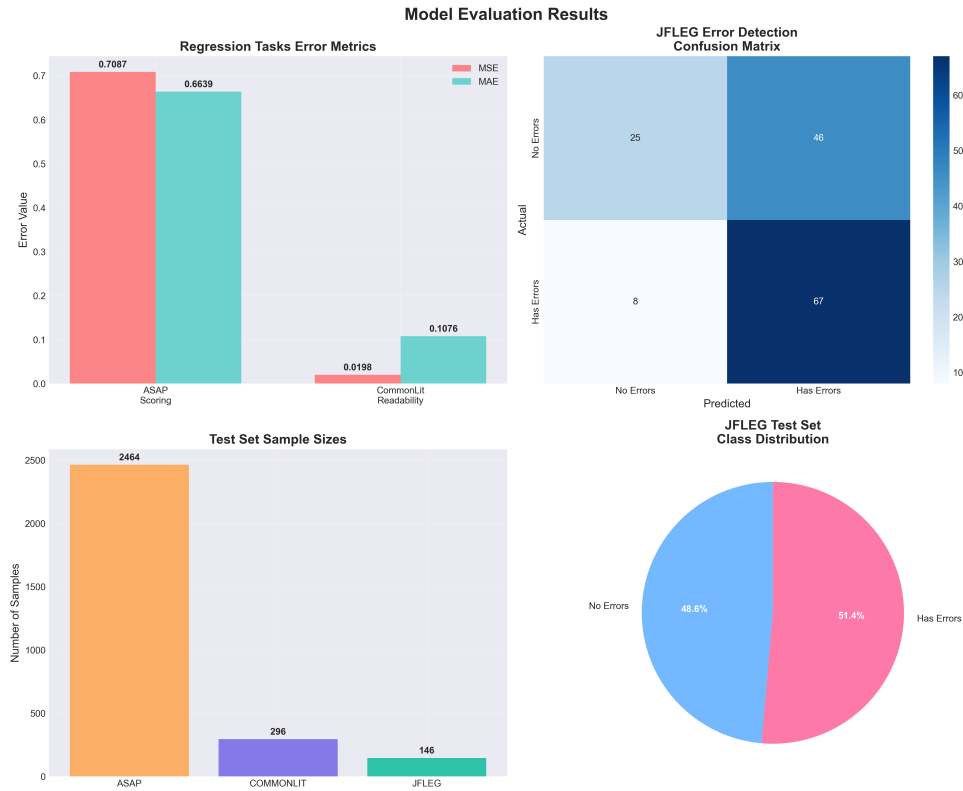
Classification Report:

	precision	recall	f1-score	support
No Errors	0.50	0.52	0.51	71
Has Errors	0.53	0.51	0.52	75
accuracy			0.51	146
macro avg	0.51	0.51	0.51	146
weighted avg	0.51	0.51	0.51	146

The obtained results suggest that the model is not able to effectively learn the grammatical rules required for this task, which may be due to the limited representativeness of the data or the overly simple MLP architecture.

5.4.2 CNN

The evaluation of the multi-head CNN model was performed on the same three tasks: essay scoring (ASAP), readability prediction (CommonLit), and grammatical error detection (JFLEG). Each task was evaluated using appropriate regression or classification metrics.



Rysunek 2: CNN Evaluation Results

ASAP (Essay Scoring – Regression Task) The CNN model achieved the following results in the essay scoring task:

- Number of samples: 2464
- Mean Squared Error (MSE): 0.7087
- Mean Absolute Error (MAE): 0.6639
- Root Mean Squared Error (RMSE): 0.8419
- Score range: 1.00 -- 6.00

Compared to the MLP model, CNN achieved noticeably lower error values across all metrics, indicating improved performance. This suggests that the CNN architecture is more effective in capturing local syntactic and lexical features relevant for scoring.

CommonLit (Readability Prediction – Regression Task) For readability prediction, the CNN model demonstrated strong performance:

- Number of samples: 296
- Mean Squared Error (MSE): 0.0198
- Mean Absolute Error (MAE): 0.1076
- Root Mean Squared Error (RMSE): 0.1405
- Readability range: 0.04 -- 0.96

While slightly worse than the MLP in this task, the CNN model still achieved low error rates, confirming its capacity to generalize over readability patterns.

JFLEG (Grammatical Error Detection – Classification Task) The CNN model outperformed the MLP in classifying grammatical errors:

- Number of samples: 146
- Accuracy: 0.6301
- Class distribution:
 - No errors (label 0): 71
 - Has errors (label 1): 75

Confusion Matrix:

	Predicted: No Errors	Predicted: Has Errors
Actual: No Errors	25	46
Actual: Has Errors	8	67

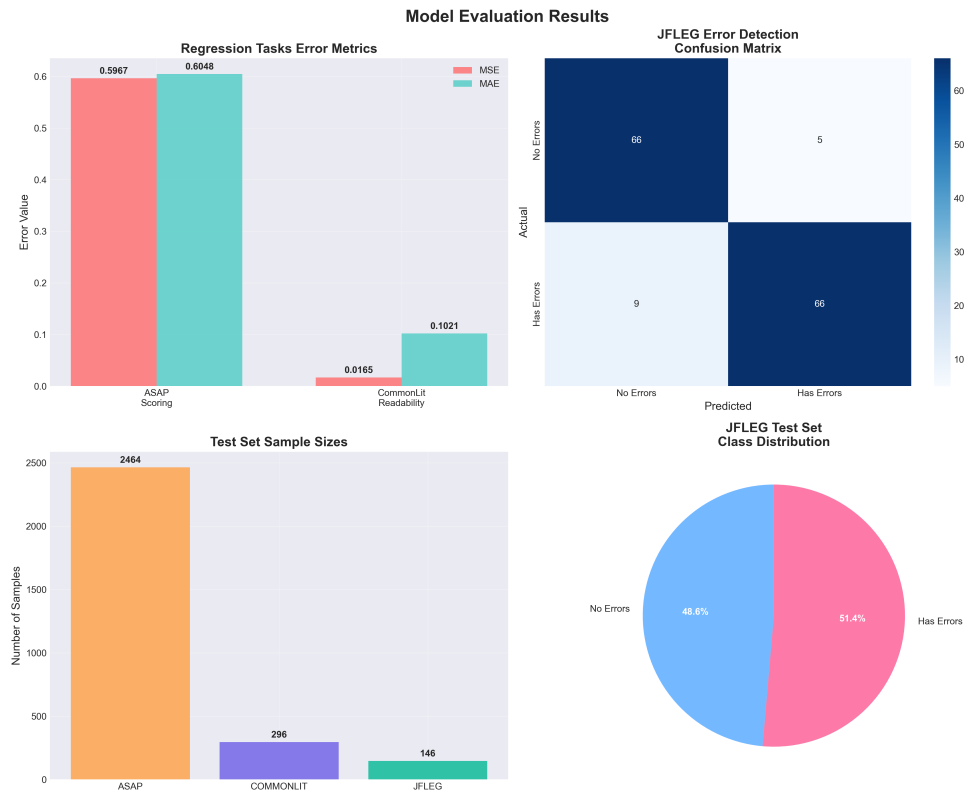
Classification Report:

	precision	recall	f1-score	support
No Errors	0.76	0.35	0.48	71
Has Errors	0.59	0.89	0.71	75
accuracy			0.63	146
macro avg	0.68	0.62	0.6	146
weighted avg	0.67	0.63	0.6	146

The CNN model achieved much higher recall for detecting errors (label 1), showing improved sensitivity to incorrect grammatical constructions. However, the low recall for correct sentences (label 0) indicates a trade-off, where the model tends to over-predict errors.

5.4.3 RNN

The multi-head RNN model was also evaluated across the same three tasks, yielding the best overall performance among all tested architectures.



Rysunek 3: RNN Evaluation Results

ASAP (Essay Scoring – Regression Task) The RNN model produced the most accurate essay score predictions:

- Number of samples: 2464
- Mean Squared Error (MSE): 0.5967
- Mean Absolute Error (MAE): 0.6048
- Root Mean Squared Error (RMSE): 0.7724
- Score range: 1.00 -- 6.00

These are the lowest errors recorded among the three models. The RNN’s ability to model sequential dependencies seems particularly beneficial for essay scoring, where the coherence and flow of text play a critical role.

CommonLit (Readability Prediction – Regression Task) In the readability prediction task, the RNN model again delivered the best results:

- Number of samples: 296
- Mean Squared Error (MSE): 0.0165
- Mean Absolute Error (MAE): 0.1021
- Root Mean Squared Error (RMSE): 0.1285
- Readability range: 0.04 -- 0.96

The low error values suggest excellent model generalization, likely due to the RNN’s capacity to account for sentence structure and information progression.

JFLEG (Grammatical Error Detection – Classification Task) The RNN model significantly outperformed both MLP and CNN in this binary classification task:

- Number of samples: 146
- Accuracy: 0.9041
- Class distribution:
 - No errors (label 0): 71
 - Has errors (label 1): 75

Confusion Matrix:

	Predicted: No Errors	Predicted: Has Errors
Actual: No Errors	66	5
Actual: Has Errors	9	66

Classification Report:

	precision	recall	f1-score	support
No Errors	0.88	0.93	0.9	71
Has Errors	0.93	0.88	0.9	75
accuracy			0.9	146
macro avg	0.9	0.9	0.9	146
weighted avg	0.91	0.9	0.9	146

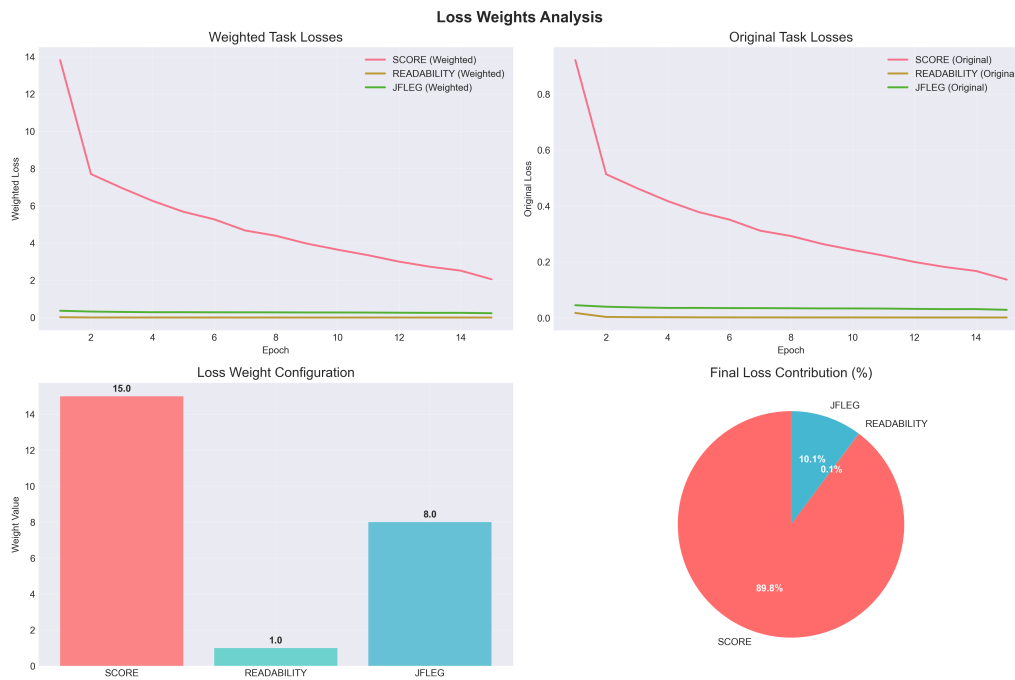
The high scores across all classification metrics confirm the RNN’s ability to capture the sequential context of grammar, outperforming both CNN and MLP by a large margin.

6 Results

6.1 MLP

6.1.1 Training Dynamics and Loss Weighting

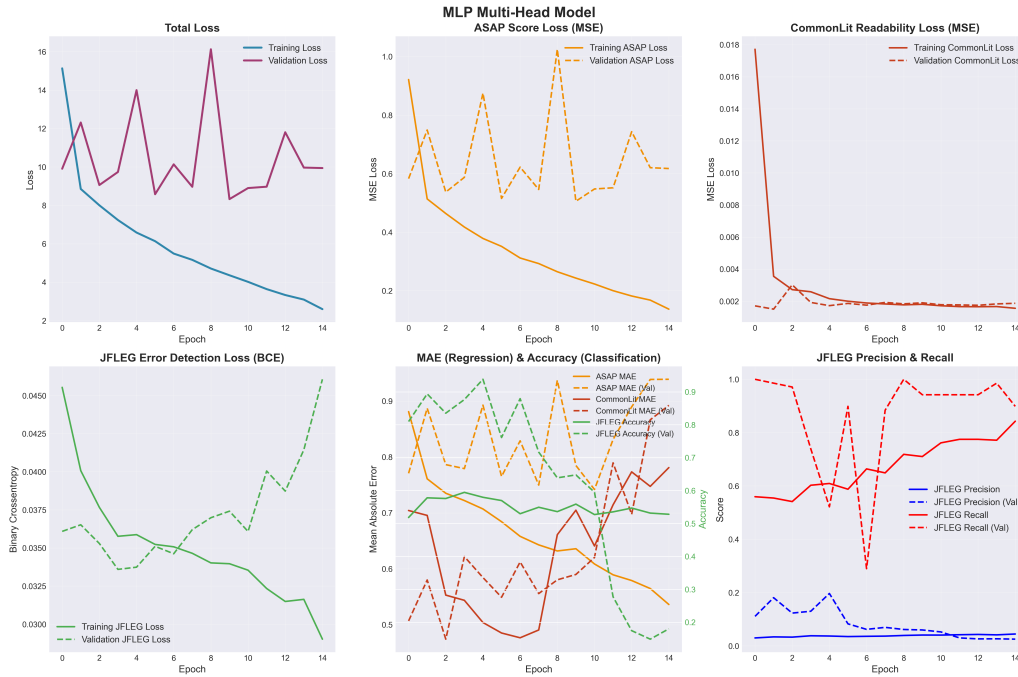
The multi-head MLP model was trained with a batch size of 32. Although the training was set to run for 50 epochs, it stopped early after 15 epochs. This was likely due to early stopping based on validation loss. As we can see the figure below shows the weights used for the loss functions of each task. The task of essay scoring (SCORE) had the highest weight (15.0), meaning it was the most important during training. The JFLEG task (grammar error detection) had a medium weight (8.0), and the readability task had the smallest weight (1.0). Because of this, almost 90% of the total loss came from the SCORE task, 10% from JFLEG, and only 0.1% from readability.



Rysunek 4: MLP Loss Weights Analysis

6.1.2 Loss Evolution and Task Performance

Figure 5 shows the training and validation losses for each task, along with regression (MAE) and classification (accuracy, precision, recall) metrics.



Rysunek 5: MLP Training Results

6.1.2.1 ASAP Score Prediction (Regression): The model learned this task quite well during training | the loss went down. However, the validation loss was unstable and sometimes increased, which may mean the model was starting to overfit. Still, the final validation MAE was not too high.

6.1.2.2 CommonLit Readability (Regression): Even though this task had a very small weight, the model still learned it well. The losses (MSE and MAE) stayed low and stable, both during training and validation. This suggests the task was easier and didn't need much weight to train properly.

6.1.2.3 JFLEG Error Detection (Classification): The training loss for this task went down, but the validation loss was less stable and increased slightly toward the end. Accuracy on the validation set stayed around 0.6. The model had high recall (it detected most errors), but low precision (it also made many false positives). This means the model tended to say there was an error even when there wasn't one.

6.1.3 Total Loss and General Trends

Total training loss steadily decreased across epochs, indicating convergence. Validation loss, however, showed instability, which is likely a result of high variance in the SCORE task and possible imbalance between tasks caused by the large loss weights.

6.1.4 Summary

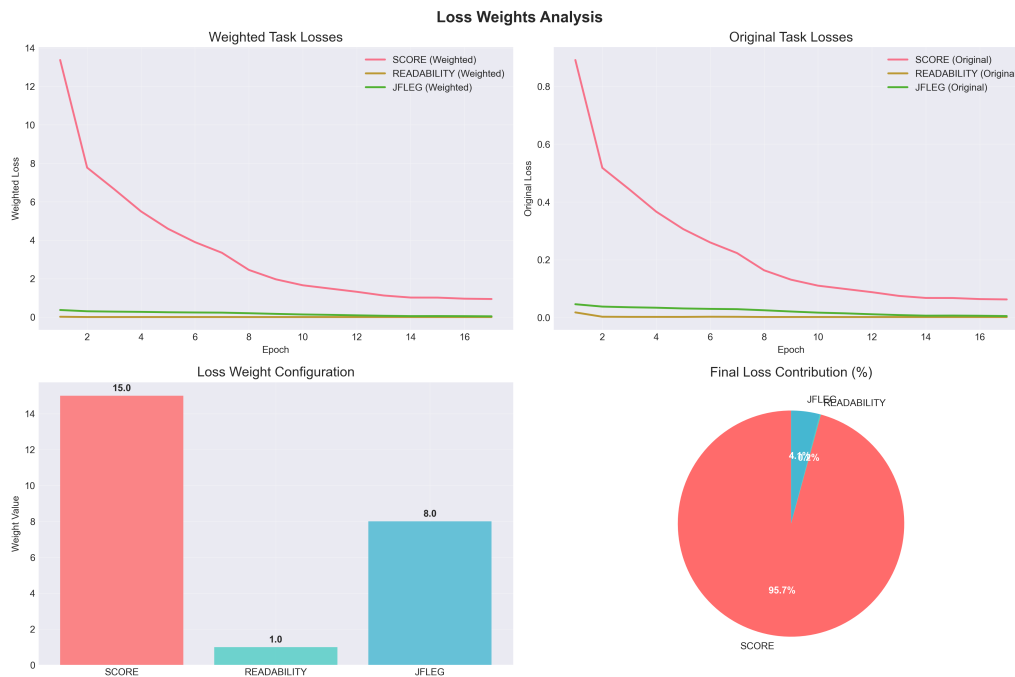
The model was able to learn all three tasks simultaneously, but task performance was highly influenced by the loss weighting strategy. While the SCORE task showed major influence on the training dynamics, both CommonLit and JFLEG tasks demonstrated the capacity to learn despite having relatively low weight contributions.

6.2 CNN

6.2.1 Training Dynamics and Loss Weighting

The multi-head CNN model was trained with a batch size of 32 and an early stopping mechanism. The training ran for 17 epochs before stopping. As with the MLP, the loss weighting configuration emphasized the essay scoring (SCORE) task, which received the highest loss weight (15.0). The grammatical error detection (JFLEG) task had a medium weight (8.0), and the readability task had

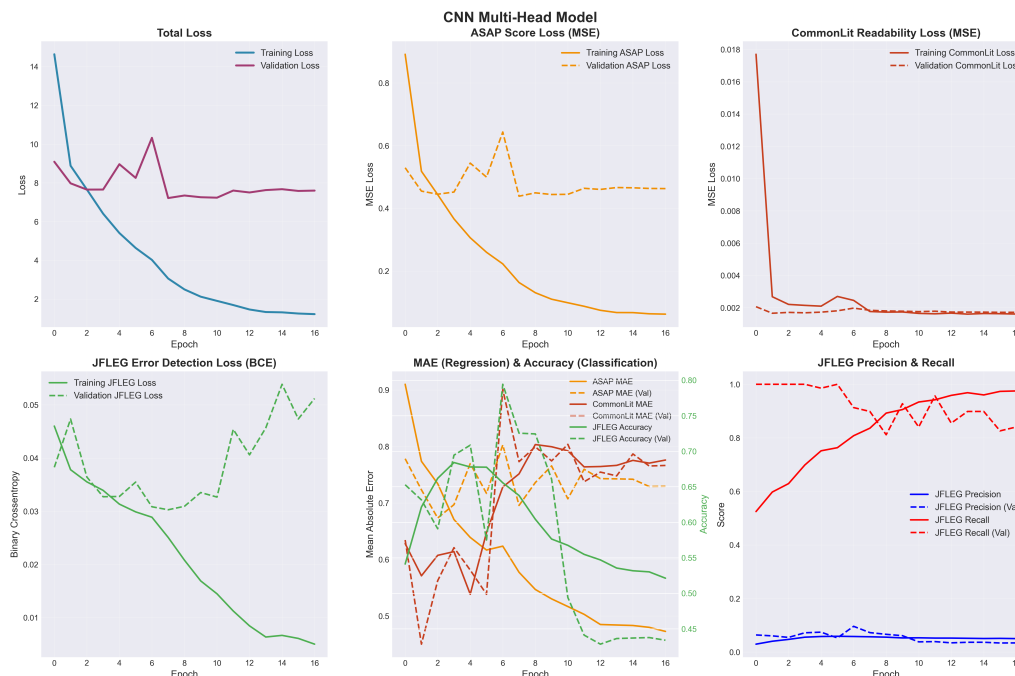
the smallest weight (1.0). This configuration ensured that SCORE dominated the total loss during training.



Rysunek 6: CNN Loss Weights Analysis

6.2.2 Loss Evolution and Task Performance

Figure 7 shows the training and validation losses per task, as well as regression and classification metrics during training.



Rysunek 7: CNN Training Results

6.2.2.1 ASAP Score Prediction (Regression): The CNN model showed consistent learning on the essay scoring task. The training and validation losses decreased smoothly, with final metrics outperforming the MLP model. The CNN's ability to extract local patterns likely helped improve regression accuracy.

6.2.2.2 CommonLit Readability (Regression): Although the readability task was low in weight, the CNN still maintained stable and low losses throughout training. The results suggest that this task benefited from general linguistic features learned from the other tasks.

6.2.2.3 JFLEG Error Detection (Classification): The CNN model achieved better classification accuracy than the MLP. It showed high recall for detecting grammatical errors but significantly lower recall for identifying grammatically correct sentences. This imbalance implies a tendency to overpredict errors, resulting in lower precision and an imbalanced confusion matrix.

6.2.3 Total Loss and General Trends

Total loss decreased steadily during training, indicating good convergence. However, similar to MLP, the validation loss fluctuated due to the SCORE task’s dominant influence and the difficulty of balancing three tasks with different objective types.

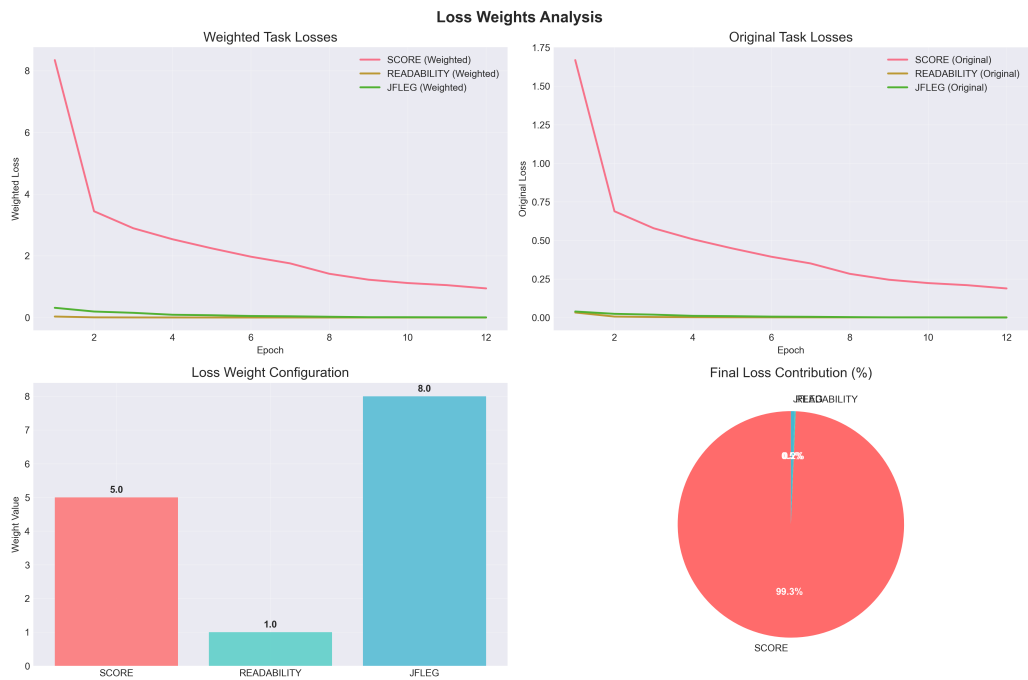
6.2.4 Summary

The CNN model showed improved regression and classification performance over the MLP. It was especially more effective in the JFLEG task, although it still struggled with false positives. Overall, the architecture’s ability to capture local dependencies gave it an advantage, especially in detecting structural errors and rating essays.

6.3 RNN

6.3.1 Training Dynamics and Loss Weighting

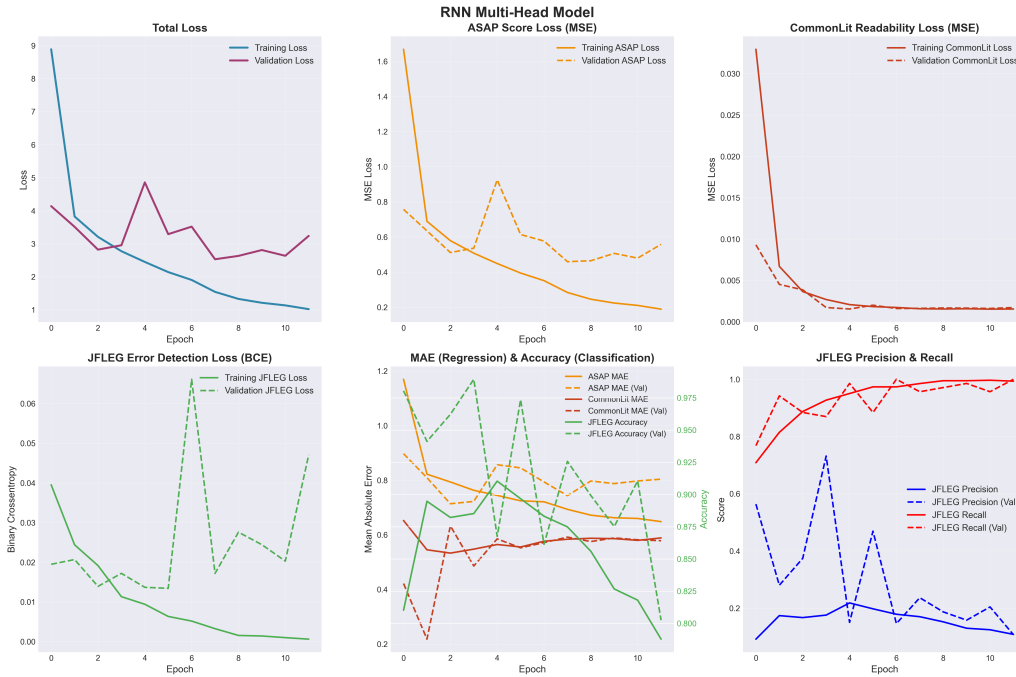
The multi-head RNN model was trained with a batch size of 32 and stopped early after 12 epochs. Unlike the MLP and CNN models, which assigned the highest loss weight to the SCORE task, the RNN used a more balanced configuration: grammatical error detection (JFLEG) received the highest weight (8.0), essay scoring (SCORE) was moderately weighted (5.0), and readability remained the least prioritized task (1.0). As a result, training focused more heavily on optimizing the classification task (JFLEG), which may explain the significantly better performance in this area compared to other models.



Rysunek 8: RNN Loss Weights Analysis

6.3.2 Loss Evolution and Task Performance

Training and validation dynamics are illustrated in Figure 9, which presents the evolution of losses and metrics for each task.



Rysunek 9: RNN Training Results

6.3.2.1 ASAP Score Prediction (Regression): Despite receiving a lower loss weight than in MLP or CNN, the RNN achieved the best results for this task. The model converged quickly and maintained low validation loss, suggesting strong generalization. This implies that even with moderate training emphasis, the RNN was effective at capturing structural and semantic patterns in essay text.

6.3.2.2 CommonLit Readability (Regression): The readability task once again had the smallest weight (1.0), yet the RNN handled it remarkably well. Both training and validation errors were minimal and stable, indicating that this task did not require significant weighting to be learned effectively | likely due to predictable linguistic cues related to readability.

6.3.2.3 JFLEG Error Detection (Classification): The RNN clearly outperformed both MLP and CNN on this task. Its classification accuracy exceeded 90%, with high precision and recall for both grammatically correct and incorrect sentences. The confusion matrix showed a balanced prediction pattern, suggesting that the higher weight given to this task (8.0) helped the model focus effectively on learning grammatical distinctions.

6.3.3 Total Loss and General Trends

Overall training loss steadily decreased, while validation loss remained stable with less fluctuation than in the MLP or CNN cases. This suggests that the RNN architecture, combined with a more balanced loss weighting scheme, led to smoother convergence and better task alignment, especially for JFLEG.

6.3.4 Summary

The RNN model demonstrated the strongest overall performance among the tested architectures. Its strength in modeling sequential data allowed it to outperform other models in all three tasks, particularly in grammatical error detection. The revised loss weighting further amplified the model's classification capabilities without degrading its regression performance, highlighting RNNs as a powerful backbone for multitask Neuro-linguistic programming settings.

7 Discussion

In this project, we compared three different neural network architectures (MLP, CNN, and RNN) in a multi-task learning setup to see how well they perform on different natural language processing tasks: essay scoring (ASAP), readability prediction (CommonLit), and grammar error detection (JFLEG).

7.1 Performance Comparison Across Tasks

7.1.1 ASAP

This regression task benefits from models that can capture both lexical patterns and higher-level coherence. The RNN model achieved the best rates, outperforming both CNN and MLP. The results suggest that the sequential modeling capabilities of RNNs allow better understanding of essay structure, which is critical for accurate scoring. CNNs also performed well, likely due to their ability to extract n-gram features, while MLP lagged behind, indicating its limited capacity to model text complexity.

7.1.2 CommonLit

In this regression task, all models achieved high accuracy, but again, the RNN delivered the best performance (RMSE = 0.1285), slightly surpassing MLP (RMSE = 0.1330) and CNN (RMSE = 0.1405). This suggests that readability features | often influenced by sentence structure and logical flow | are best captured by models that retain sequential dependencies. The relatively small dataset size (296 samples) may have favored models with better generalization.

7.1.3 JFLEG

This binary classification task proved to be the most challenging, especially for the MLP model, which performed only slightly above random (accuracy = 0.5137). CNN improved on this (accuracy = 0.6301), but still demonstrated a tendency to over-predict the "error" class. In contrast, the RNN model showed a significant performance boost (accuracy = 0.9041), effectively distinguishing between grammatically correct and incorrect sentences. The confusion matrix confirms the RNN's balanced performance across both classes.

7.2 Architectural Insights

The results highlight the importance of selecting model architectures aligned with the nature of the task:

- MLP - simple and quick to train, but it can't really understand the context or word order, so its performance is weaker, especially in tasks that require understanding grammar.
- CNN - they can learn local patterns (like n-grams), which works quite well for regression tasks and okay for classification. But they still struggle with longer-range dependencies in text.
- RNN - perform the best in all tasks because they can keep track of context throughout the whole sequence. That makes them a good choice for more complex language tasks like essay scoring or error detection.

8 Conclusion

This study demonstrates that RNN are highly effective in multi-task setups involving tasks on natural language processing(NLP), outperforming both MLP and CNN models. The results validate the importance of sequence modeling for language tasks that depend on context and syntax. While MLPs may still serve as strong baselines for simpler tasks, real-world applications would benefit more from sequence-aware architectures.